

DOES THE DOCUMENTATION OF DESIGN PATTERN INSTANCES IMPACT ON SOURCE CODE COMPREHENSION?

Carmine Gravino^{}, Michele Risi^{}, Giuseppe Scanniello^{}, and Genoveffa Tortora^{}



University of Salerno



University of Basilicata

Design patterns (DPs)

“A design pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice” (Alexander et al.)

This definition also holds for design patterns in object-oriented software (Gamma et al.)

The core of both kinds of design patterns is a solution to a problem in a given context.

C. Alexander, S. Ishikawa, M. Silverstein, M. Jacobson, I. Fiksdahl-King, and S. Angel. A Pattern Language - Towns, Buildings, Construction. Oxford University Press, 1977.

E. Gamma, R. Helm, R. Johnson, and J. Vlissides. Design Patterns: Elements of Reusable Object Oriented Software. Addison-Wesley, 1995.

Design Patterns in OO Software

What is a design pattern?

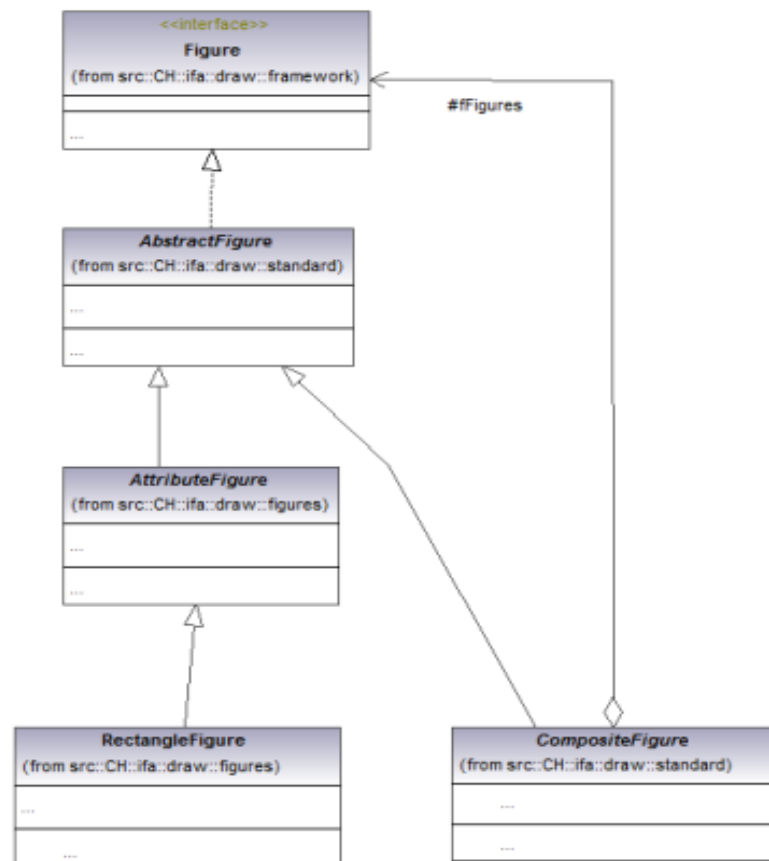
4 essential elements: name, problem, solution, consequences

Some examples, intent, and so on ...

“DPs can improve the documentation and maintenance of existing systems by furnishing an explicit specification of class and object interactions and their underlying intent” (Gamma et al.)

Our focus

DP's solutions: Instances (also known as design motifs)



Graphically Documented (GD)

We added this block

```
1: /*
2:  * @(#)CompositeFigure.java 5.1
3:  *
4:  * Composite Pattern
5:  * Component: Figure
6:  * Composite: CompositeFigure
7:  * Leaf: RectangleFigure
8:  *
9:  */
10:
11: package CH.ifa.draw.standard;
12:
13: ....
14:
15: public abstract class CompositeFigure
16:     extends AbstractFigure
17:     implements FigureChangeListener {
```

Textually Documented (TD)

Two controlled experiment

Involving students of the Master program in Computer Science

UniBas (original experiment): University of Basilicata

UniSa: (replication): University of Salerno

Two controlled experiment

Involving students of the Master program in Computer Science:

UniBas (original experiment): University of Basilicata

UniSa: (replication): University of Salerno

Participants worked on a modified version of JHOTDRAW

Dependent variables: Effort, Comprehension, Efficiency

One factor with two treatments design:

control group: using the source code with no documented design pattern instances

treatment group: using source code with the documentation (graphically or textually)

Details in: **2011_Does the Documentation of Design Pattern Instances Impact on Source Code Comprehension?.pdf** (see the eLearning platform)

DO PROFESSIONAL DEVELOPERS BENEFIT FROM DESIGN PATTERN DOCUMENTATION? A REPLICATION IN THE CONTEXT OF SOURCE CODE COMPREHENSION

Carmine Gravino^{}, Michele Risi^{}, Giuseppe Scanniello^{}, and Genoveffa Tortora^{}



University of Salerno



University of Basilicata

Issue

Do DPs improve documentation and maintenance of existing systems?

Only few evaluations have been conducted on the practical benefit of explicitly reporting DP

No study involving professionals

C. Gravino, M. Risi, G. Scanniello, and G. Tortora. Does the documentation of design pattern instances impact on source code comprehension? Results from two controlled experiments. In Procs. of the Working Conference on Reverse Engineering, pages 67-76. IEEE CS, 2011.

L. Prechelt, B. Unger-Lamprecht, M. Philippsen, and W. Tichy. Two controlled experiments assessing the usefulness of design pattern documentation in program maintenance. IEEE Trans. Softw. Eng., 28(6):595-606, 2002.

Replication with Professionals

Analyse the use of graphical and textual documentation for design pattern instances
for the purpose of evaluating them
with respect to the source code comprehension
from the point of view of project manager,
in the context of professional software developers

Context selection

25 Italian software professionals (a laboratory session for each company)

Laboratory sessions under controlled conditions (supervisors were the same in each session)

Info from a pre-questionnaire

Junior (10): working experience from 1 to 3 years

Senior (15): working experience more than 3 years

Design pattern development experience from *Low* to *Medium*

Scale: {*Very low, Low, Medium, High, Very high*}

Experimental object

A vertical slice of JHotDraw v5.1

A non trivial number of instances

Popular instances

Public dataset

1236LOCs, 26 Classes, 823 Lines of comments

Comments were translated

Control group – references to
instances were removed both
in the comment and the
identifiers

# State Pattern	2
# Adapter Pattern	1
# Strategy Pattern	1
# Decorator Pattern	1
# Composite Pattern	1
# Observer Pattern	1
# Command Pattern	1
# Template Method Pattern	1
# Prototype Pattern	1

The screenshot shows the P-MART website. On the left is a navigation menu with links like 'Portale', 'The Chair', 'Research', 'Roadmap', 'Collaborations', 'Resources', 'Downloads', 'Pidej Demo', 'Pidej in Japanese', 'Pidej Blackbox', 'P-MART', 'Replications', 'Organisation', 'Team Life', 'Teaching', and 'Links'. Above the menu is a calendar for October 2011. The main content area is titled 'P-MART' and includes a search bar, a description of the repository, and a table of stored micro-architectures.

Programs (zipped source code)	Sizes (in bytes)	Programming languages	Repository (zipped XML file)
1 - QuickXML 2001	787657	Java	Repository (Summary)
2 - Levi v0.1.1 jshdy	838103	Java	
3 - JRefactory v2.6.24	3973569	Java	
4 - Netbeans v1.0.4	37308254	Java	
5 - JUnit v3.7	792782	Java	
6 - JHotDraw v5.1	819088	Java	
8 - MapperXML v1.9.7	2524977	Java	
10 - Nutch v0.4	3023193	Java	
11 - PMD v1.8	6977806	Java	

11 - PMD v1.8	6977806	Java
10 - Nutch v0.4	3023193	Java
8 - MapperXML v1.9.7	2524977	Java
6 - JHotDraw v5.1	819088	Java
5 - JUnit v3.7	792782	Java

Variables

Dependent variables:

Effort – time to accomplish a task

Comprehension – measures the comprehension achieved
by participants

Efficiency – $\text{Comprehension} / \text{Effort}$

Evaluating the answers to a comprehension
questionnaire composed of 14 open questions and
time to accomplish the task

Variables (2)

Independent variable: Method

a nominal variable assuming values in {GD, TD, SC}

GD and TD participants were also grouped in:

DPCI (Design Patterns Correctly Identified) and DPnCI
(Design Patterns not Correctly Identified)

To understand whether different design pattern instances
affect source code comprehension

Hypotheses Formulation and Experiment design

Hn0_D_X. The participants who used D design pattern instances did not achieve significantly better results in terms of X than the participants who used source code alone (SC)

X can be Effort, Comprehension, or Efficiency

D can be GD or TD

One-tailed (we expected a positive effect of design pattern instances)

Hn1_X. There was not a significant difference with respect to X when participants used GD or TD

Two-tailed (we could not postulate any effect of GD or TD)

One factor with three treatments design: the participant working experience (i.e., the amount of years as professional developers) was the blocking factor:

9 (4 juniors and 5 seniors) to GD and 8 (3 juniors and 5 seniors) to TD and SC

Experimental tasks

Comprehension task: questions to assess several aspects related to the comprehension of the source code.

Q2. Indicating the class/es and the method/s in charge of creating, drawing, and updating the instances of the class Figure?				
How much do you trust your answer ⁺ ?				
☐ Unsure	☐ Not sure enough	☐ Sure Enough	☐ Sure	☐ Very Sure
How do you assess the question ⁺ ?				
☐ Very difficult	☐ Difficult	☐ On average	☐ Simple	☐ Very Simple
What is the source of information used to answer the question ⁺ ?				
☐ Previous Knowledge (PK)		☐ Internet (I)	☐ Source Code (SC)	
⁺ Mark only one answer				

Fig. 2. A question example from the comprehension questionnaire

Post-experiment task: to obtain feedback about the participants' perceptions of the experiment execution

Experimental and analysis procedure



In the experiment:

- Introductory lesson (instruction on the experiment)

- Execution of the comprehension and post-experiment tasks

 - No time limit to perform the tasks

 - Each participant used a PC with an Internet connection and UltraEdit to surf source code

R for statistical analysis:

- Descriptive statistics to summarize the results

- Mann-Whitney test to test the hypotheses (5% of committing Type-I-error), Effect size to obtain the standardized difference, and Statistical Power

Differences and Similarities with our previous study

Variations introduced to mitigate as many threats to validity as possible and to improve the material and the data analysis

Participants: more experienced

Experiment design: two treatment groups (GD and TD)

Group composition: The professional experience is the blocked factor

Data analysis: more analysis on the effect of the documentation type

Training session: not performed

Experimental procedure: the use of Internet

Preserved choices: **Dependent variables** and **Experimental objects**

Results: Descriptive statistics

	GD			TD			SC		
	Mean	Median	Std. Dev.	Mean	Median	Std. Dev.	Mean	Median	Std. Dev.
Effort	132	142.2	31.17	139	136.4	37	151	147.4	42.81
Comprehension	50.91	51.08	10.32	53.43	53.49	7.26	40.56	39.97	9.63
Efficiency	0.37	0.37	0.11	0.37	0.42	0.14	0.29	0.31	0.14

GD

	DPCI			DPnCI		
	Mean	Median	Std. Dev.	Mean	Median	Std. Dev.
Effort	9	10.36	3.93	8	9.13	3.67
Comprehension	80	73	27.7	47	39.13	36.88
Efficiency	8.17	8.24	4.60	4.4	5.28	6.18

TD

	DPCI			DPnCI		
	Mean	Median	Std. Dev.	Mean	Median	Std. Dev.
Effort	10	9.61	3.41	10	9.97	3.93
Comprehension	73	67.06	30.88	45	38.50	36.13
Efficiency	7.69	8.46	5.91	3.94	5.76	8.08

Results: hypotheses testing

Hn0_GD_Comprehension and ***Hn0_TD_Comprehension*** can be rejected (p-values: 0.033 and 0.009) with a large effect size ($d=1.113$ and 1.586) and high statistical power ($t=0.966$ and 0.875).

The participants who used the documentation of design pattern instances significantly better comprehended source code than those provided with source code alone.

Hn0_D_Effort and ***Hn0_D_Efficiency*** cannot be rejected

Hn1_X: cannot be rejected. For all the dependent variables the difference between the participants who used GD and TD design pattern instances is not statistically significant.

Main Findings

The mean improvement achieved with the design pattern instances was:
25.5%, when graphically documented
31.9%, when textually documented
(professionals were more comfortable with source code than students)

The time to perform the comprehension task did not increase
(significantly) w.r.t. the use of the source code alone
Unexpected, considering the further documentation to read and interpret
(more evident for GD)

Identification of DP instances is crucial:

the average Comprehension values achieved by DPCI participants is 70.2%
greater than that of DPnCI participants for GD. For TD is 62.2%

GD participants indicated DP as first source of information, while TD the
source code (DP instances considered integral part of the code)

Post Experiment Questionnaire

Time appropriate

Objectives clear

Source code clear

Comment clear

The documentation of the instances is useful

Threats to validity

Conclusion validity

Non-parametric statistical tests for unpaired analysis

Internal validity

Fatigue effect was mitigated giving a break between subtasks

The exchange of information was prevented

Construct validity

The measures for dependent variables are widely used

The modification of the identifiers and comment

External validity

The system used (open source and developed with DP)

Italian professionals (mostly junior)

Pieces of Evidences

The use of design patterns can increase maintainability

Correct identification of DP is crucial

The correct identification of instances is more relevant than the kind of documentation

No difference between Graphically and Textually documented instances

→ It is important to provide means to effectively identify instances

Our previous experiments

UniBas: 24 MhS in Computer Science at the University of Basilicata

UniSa: 17 MhS in Computer Science at the University of Salerno

Control group – source code alone (UniBas and UniSa)

Treatment group – source code + DP instances (graphically documented in UniBas and textually documented in UniSa)

One factor with two treatment: each participant used either DP (group A) or SC (group B)

Blocking factor Ability - high and low ability participants were equally distributed in the groups

Prev. Experim.: Main Findings

Better results were achieved on DP

The effect is significant on Effort and Efficiency for UniBas

When using textually documented (wrt graphically) instances the time is significantly less

When instances are correctly identified a significant difference was observed for all the variables

Except for Effort in UniBas

The participants in UniSa and UniBas indicated that they trusted the explicitly reported design pattern instances and found them useful

Fmeasure

(1) $A_{s,i}$ as the set of string items provided as answer to the question i by the participant s ; (2) C_i as the correct set of items expected for the question i (i.e., the oracle⁴). For each answer, we can compute:

$$precision_{s,i} = \frac{|A_{s,i} \cap C_i|}{|A_{s,i}|} \quad recall_{s,i} = \frac{|A_{s,i} \cap C_i|}{|C_i|}$$

Precision (i.e., the fraction of items in the answer that are correct) and recall (i.e., the fraction of correct items in the answer) measure the correctness of the answers to a given question and the completeness of the answers, respectively. To get a balance between correctness and completeness, we used a standard aggregated measure based on the combination of precision and recall:

$$F-Measure_{s,i} = \frac{2 \cdot precision_{s,i} \cdot recall_{s,i}}{precision_{s,i} + recall_{s,i}}$$